Assignment: Smarter Retrieval with Query Expansion & Reranking

Build a RAG pipeline over your own PDFs, then prove which technique actually helped — and why.

Level: Beginner – Intermediate | **Est. time: 3–4 hours** | **Stack: LangChain · FAISS · OpenAI** | **Deliverable: 1 report + screenshots**

> **The one-line goal.** You already have a working RAG pipeline. In this assignment you will point it at *your own* set of PDFs, run it one stage at a time, and carefully observe how two techniques — **query expansion** (finding more of the right stuff) and **reranking** (putting the best stuff on top) — change what the system retrieves and answers. You will not write new code. Your job is to **run, observe, tune, and explain.**

What you're working with (read this first)

A RAG ("Retrieval-Augmented Generation") system answers a question by first *fetching* the most relevant pieces of your documents, then handing those pieces to an AI model to write the answer. If the fetching step brings back the wrong pieces, the answer will be wrong — no matter how good the AI model is. This assignment is entirely about making that fetching step smarter.

Three ideas you will see in action. Keep this table open while you work — every task below maps back to it.

Idea	What it optimises	Plain-English meaning
Query expansion	Recall (finding more)	The same question can be phrased many ways. "Profit after tax", "PAT", "net earnings", "bottom line" all mean the same thing. Expansion asks an AI to rewrite your question several ways, searches with *all* of them, and pools the results — so a good chunk isn't missed just because you didn't use the document's exact words.
Reranking	Precision (finding the best)	Pooling many searches also drags in some weak, off-topic chunks. A reranker re-reads every pooled chunk *together with* your original question and gives each one a fresh relevance score, so the genuinely best pieces float to the top and the noise sinks.
Rank application	Ordering (what changed)	The reranker doesn't just filter — it *reorders*. A chunk sitting at position #7 after search might jump to #1 after reranking; another might fall and get dropped. Watching this "before vs after" movement is how you *see* reranking doing its work, rather than taking it on faith.

Setup (do this once, before the tasks)

1. Clone or download the project folder and open it in your editor. Confirm you can see these files: `ingest.py`, `retriever.py`, `query_expansion.py`, `reranker.py`, `answer_chain.py`, and `requirements.txt`.
2. Create and activate a fresh virtual environment, then install everything with the requirements file provided. (One command; it pulls in LangChain, FAISS, the OpenAI libraries, and the reranker library.)
3. Create a file named `.env` in the project folder and put your `OPENAI_API_KEY` inside it. The scripts read the key from here automatically — you never paste it into the code.
4. **Swap in your own documents.** Delete the three sample PDFs that ship with the project and drop *your own* set of PDFs into the same folder. Anything text-based works: quarterly reports, annual reports, a product manual, research papers, policy documents. Use 3–6 PDFs so there's enough variety to make retrieval interesting. Note down what your documents are — you'll need this for the report.

> **Choosing good documents matters.** Pick PDFs that (a) actually contain text you can select and copy — not scanned images, and (b) share a theme so questions can plausibly pull from more than one file. A folder of three years of annual reports for one company is ideal: it lets you ask questions that span documents, which is exactly where expansion and reranking earn their keep.

The golden rule of order

> Run the stages **in the sequence below**. Each stage depends on the one before it. In particular, `ingest.py` builds the searchable index that every other file reads — if you skip it, everything else fails with a "file not found" error pointing you right back here. Run `ingest.py` first, and re-run it *only* when you change your PDFs.

Task 1 — Build the index (ingestion)

RUN 1st **ingest.py — turn PDFs into a searchable index**

What it does: reads every PDF, cuts each one into small overlapping chunks of text, converts each chunk into a list of numbers that captures its meaning (an "embedding"), and saves the whole collection to disk as a FAISS index. Think of it as the loading dock: raw PDFs go in, a searchable index comes out. Nothing downstream ever touches your PDFs again — they only talk to this index.

```
python ingest.py
```

What to watch for on screen: the number of PDFs found, the total pages loaded, the total **chunks** created, a preview of the first chunk, and finally the total **vectors** in the index. A new folder called `faiss_index` should appear with two files inside it.

Record in your report:

- How many PDFs, pages, and chunks did your documents produce?
- Read the sample chunk it prints. In one line: does it look like a clean, self-contained piece of text, or is it cut awkwardly mid-sentence? (You'll come back to this in the tuning task.)

Why chunk at all? An embedding can only hold so much meaning in one vector. If you embedded an entire report as a single vector, the revenue figures, the CEO's commentary, and the balance sheet would all blur into one vague point — and a search for a specific number would land nowhere useful. Small chunks keep each vector focused on one idea, so a specific question can match the exact paragraph that answers it.

Task 2 — See plain retrieval (the baseline)

RUN 2nd retriever.py — a single, ordinary search

What it does: loads the index you just built and runs *one* plain search for whatever question you type — no expansion, no reranking. This is your **baseline**: the "before" picture that the smarter techniques will be measured against.

```
python retriever.py
```

When prompted, type a real question about your documents. Ask something whose answer you can verify yourself by opening the PDF.

Do this deliberately — you'll reuse both questions all the way through:

- **Question A (plain wording):** phrase it using words you'd expect to appear *literally* in the document. Example for a financial report: "What was the total revenue in the second quarter?"
- **Question B (tricky wording):** ask the same underlying thing, but using different words or an abbreviation the document might *not* use. Example: "How much did the company keep after tax in Q2?" instead of "profit after tax".

Record in your report: for both Question A and Question B, note which chunks came back (source file + page) and whether the chunk that truly answers the question was among them. Keep a screenshot. Most people find plain retrieval handles Question A fine but stumbles on Question B — that gap is the whole reason expansion exists.

Task 3 — Watch reranking reorder the results (rank application)

RUN 3rd reranker.py — before vs after, side by side

What it does: runs a plain search for your question (like Task 2), then re-scores every retrieved chunk against your *original* question using a smarter model that reads the question and each chunk **together**. It prints two clearly separated blocks — **BEFORE** (the raw search order) and **AFTER** (reordered by true relevance) — then writes a final answer from the top chunks.

```
python reranker.py
```

The first time you run this, it downloads a small reranking model (about 80 MB). That's normal and only happens once.

This is where you observe "rank application." In the AFTER block, each chunk shows its new position, its relevance score, and a movement marker: an up-arrow means it climbed, a down-arrow means it fell, a dash means it stayed put. Chunks past the cut-off are marked **dropped**; the survivors are marked **KEPT**.

Record in your report:

- Run this for **Question B** (the tricky one). Did any chunk make a big jump — say from #6 up to #1? Note its before and after positions.
- Did the chunk that actually answers the question end up in the KEPT set? Was it there before reranking too, just lower down?
- Compare the final written answer here to what plain retrieval alone would have supported. Is it more accurate? Take a screenshot of the BEFORE/AFTER block — this is your headline evidence.

Why the reranker is more accurate. Plain search compares pre-computed vectors — fast, but the question and the chunk were never actually read together. The reranker reads each pairing directly, word for word, so it judges relevance far more sharply. The trade-off: it's slower, which is exactly why we only use it on the short list of already-retrieved chunks, never across the whole index.

Recall then precision. Hold this mental picture: a wide net first (grab everything that might be relevant), a tight filter second (keep only what truly is). Reranking is the tight filter. In the next task you'll add the wide net — query expansion — in front of it, and see the two work as a team.

Task 4 — Run the full pipeline (query expansion + reranking)

RUN 4th query_expansion.py — everything together

What it does, in order:

1. **Expands** your question into several alternative phrasings using an AI model.
2. **Searches** once for the original question and once for *each* alternative phrasing.
3. **Merges** all those results into one pile and removes duplicate chunks.
4. **Reranks** the merged pile (the Task 3 technique) to keep only the best few.
5. **Answers** the question from those final chunks.

```
python query_expansion.py
```

This runs as a loop — keep asking questions, type `exit` to quit.

What to watch on screen: the tool prints the original query and every rewritten version, then for each search it shows every chunk it pulled, tagged NEW (seen for the first time) or duplicate (already collected). Finally it reports how many *unique* chunks survived the merge, then shows the same BEFORE/AFTER reranking block from Task 3.

Record in your report:

- Run **Question B** again here. Read the rewritten phrasings the AI generated — list two you found clever, and note whether any used a synonym or abbreviation your documents actually contain.
- Did expansion surface a good chunk that plain retrieval in Task 2 *missed*? This is the payoff moment — call it out explicitly with the source and page.
- Note the numbers: how many total searches ran, how many chunks were fetched in total, and how many *unique* chunks remained after de-duplication. Explain in one sentence why "total fetched" is so much larger than "unique kept".

The honest trade-off (mention this in your report). Expansion isn't free. Instead of one search you now run several, plus an extra AI call just to generate the rewrites. That means more time and more cost per question. The payoff is that you're far less likely to miss the right chunk. The right verdict is situational: expansion is worth it when missing the answer is more expensive than a slightly slower, pricier query.

Task 5 — Tuning challenge (change one knob at a time)

Now experiment. Each setting below lives at the top of a file as a named value — change it, save, re-run the relevant stage, and observe. **Change only one at a time** so you can attribute any difference to that one change. If you touch anything in `ingest.py`, you must re-run `ingest.py` to rebuild the index.

Setting (and file)	Try changing it to…	What to observe and write down
`CHUNK_SIZE` / `CHUNK_OVERLAP` ingest.py	A much smaller chunk (e.g. very short) and a much larger one. Re-run ingestion each time.	Does the total chunk count go up or down? With tiny chunks, do answers start losing context? With huge chunks, does retrieval get less precise? Which felt best for *your* documents?
`TOP_K` retriever.py	A lower number, then a higher number of chunks returned per search.	More chunks means better odds of catching the answer, but more noise for the reranker to clean up. Where's the sweet spot for your set?
`NUM_EXPANSIONS` query_expansion.py	Fewer rewrites, then more rewrites of the question.	Do more rewrites find more unique chunks, or just repeat the same ones as duplicates? Watch the "unique chunks after de-duplication" number.
`RERANK_TOP_N` query_expansion.py	Keep fewer chunks, then more chunks after reranking.	Does feeding the answer model fewer, sharper chunks improve the answer — or does it start missing detail? Does more context help or just add clutter?

Record in your report: pick the **two** changes that made the most noticeable difference for your documents, and explain what changed and why you think it happened.

Reflection questions (answer in your own words)

1. In one or two sentences each, explain to a friend who's never heard these terms: what does **query expansion** do, and what does **reranking** do? Why do we need *both* rather than just one?
2. Give a concrete example from *your own* documents where a plain search would likely miss the right chunk, but expansion would catch it. (Point to the actual wording difference.)
3. Reranking is described as "slow but accurate" and plain vector search as "fast but rough". Why do we deliberately use the fast one first and the accurate one second, instead of using the accurate one for everything?
4. You saw the same chunk appear as a "duplicate" under several different rewritten queries. Is that a bug or a good sign? Explain what it tells you.
5. If your boss said "this is too slow and too expensive, simplify it," which single technique would you drop first, and what would you lose by dropping it?

What to submit

1. A short **report** (PDF or doc) containing every "Record in your report" answer from Tasks 1–5 and all five reflection answers.
2. A one-line description of **the documents you chose** and why.
3. **Screenshots** as evidence: the ingestion summary (Task 1), the BEFORE/AFTER reranking block (Task 3), and the expansion + de-duplication output (Task 4).

You will be assessed on:

- Correct run order and a working pipeline over your own PDFs.
- Clear evidence that you *observed* expansion adding chunks and reranking reordering them — not just that you ran the files.
- Quality of explanation in the reflection answers, in plain language.

- Thoughtfulness of your tuning conclusions.

Stretch goals (optional, for extra credit)

- **Break it on purpose.** Ask a question your documents genuinely cannot answer. Does the system correctly say it doesn't know, or does it invent something? Report what happened.
- **Cross-document question.** Ask something that can only be answered by combining two different PDFs (e.g. "how did revenue change from the first report to the third?"). Does expansion + reranking pull chunks from multiple files? Show it.
- **Write the trade-off memo.** In half a page, imagine you're recommending whether to ship this pipeline for a real company help-desk. Weigh the accuracy gains against the added cost and latency, and give a clear recommendation.

Cheat sheet — keep this page open while you work

Run order at a glance

Order	Command	What it produces	Re-run when…
1st — once	`python ingest.py`	The `faiss_index` folder (your searchable index)	You add, remove, or change PDFs, or change chunk settings
2nd	`python retriever.py`	One plain search — your baseline	Any time you want a "before" comparison
3rd	`python reranker.py`	BEFORE/AFTER reordering + an answer	Any time you want to inspect reranking alone
4th	`python query_expansion.py`	The full pipeline: expand → search → merge → rerank → answer	Your main demo; type `exit` to quit

`answer_chain.py` is a helper that both `reranker.py` and `query_expansion.py` use to write the final answer — you never run it directly.

Mini-glossary

Term	In one line
Chunk	A small slice of a PDF, sized to hold one idea.
Embedding	A chunk turned into numbers that capture its meaning.
Index (FAISS)	The store of all embeddings you search against.
Retrieval	Fetching the chunks closest in meaning to a question.
Query expansion	Rewriting the question many ways to find more.
Reranking	Re-scoring fetched chunks to surface the best.
Recall vs precision	Finding *more* vs finding the *best*.

If something breaks

You see…	Do this
"FAISS index not found"	You skipped step 1 — run `ingest.py` first.
"No PDF files found"	Your PDFs aren't in the project folder. Move them in.
An authentication / API-key error	Your `.env` is missing or the key is wrong.
A long pause on first `reranker.py` run	Normal — the reranking model is downloading once.
Empty or "I could not find" answers	The chunk may be too small, or the answer isn't in your PDFs. Revisit Task 5.

> **The mental model to walk away with.** Retrieval alone is one person searching with one set of words. **Query expansion** sends several people, each asking differently, so you miss less. **Reranking** then reads everything they brought back and keeps only what truly answers *your* question, best first. Recall widens the net; precision tightens it. That partnership — not any single trick — is what makes retrieval trustworthy.